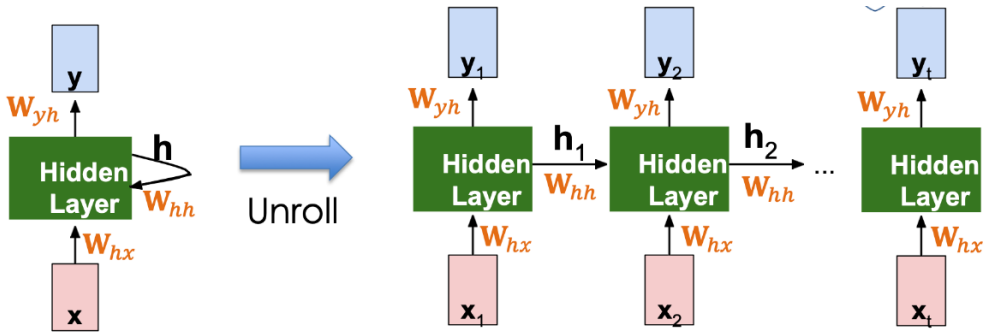
CG3201 Machine Learning and Deep Learning
Lab 4 – Recurrent Neural Network

1. Introduction



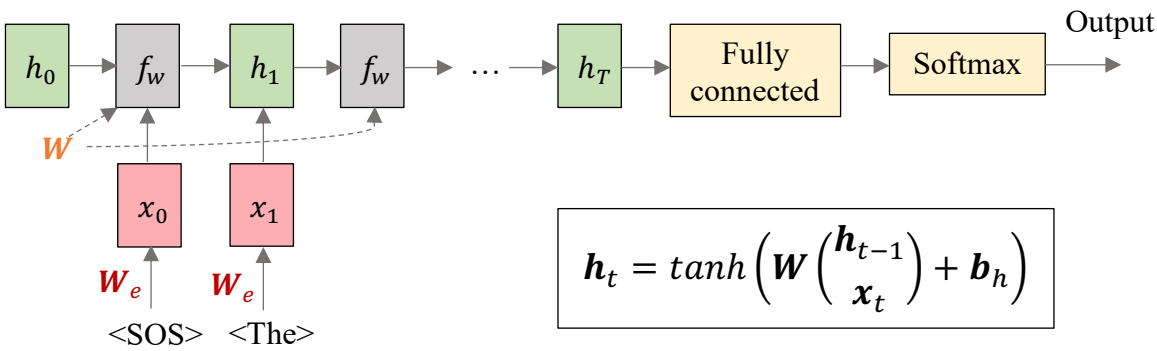
Recurrent Neural Networks (RNNs) represent a class of deep learning architectures designed to process sequential data by leveraging feedback loops. This structure is particularly effective for modelling sequences with temporal dependencies, such as natural language (text), video frames, and audio signals. In this lab, you will develop RNN models for sentiment analysis and compare their performance.

2. Dataset

For this assignment, we will utilize the IMDB Dataset, a widely recognized benchmark for binary sentiment classification. This dataset contains 50,000 highly polar movie reviews (labelled as either positive or negative) and is available on Kaggle: https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews

This dataset is partitioned into three sets: 60% for training, 20% for validation, and 20% for testing. The exploratory data analysis (EDA) of this dataset, including metrics such as average sentence length and word frequency, is provided in the template.

3. Vanilla Recurrent Neural Networks

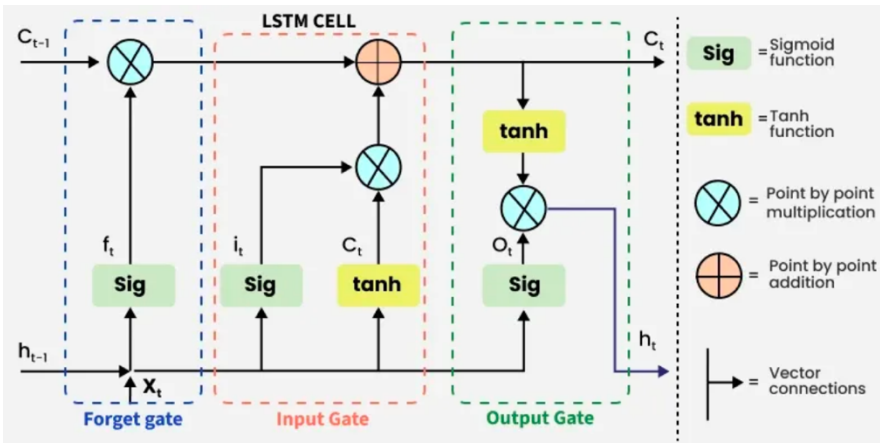


$$\boldsymbol{h}_t = tanh\left(\boldsymbol{W}\begin{pmatrix}\boldsymbol{h}_{t-1}\\ \boldsymbol{x}_t\end{pmatrix} + \boldsymbol{b}_h\right)$$

a. In this task, you are required to implement an RNN architecture featuring a single hidden layer with 128 hidden units. The final hidden state of the RNN is passed into a fully connected layer consisting of 2 output nodes to perform binary sentiment classification. You are required to build the vanilla RNN cell through both manual construction using nn.Linear layers and library implementation utilizing the standard PyTorch nn.RNN module. Train both implementations for 10 epochs using a learning rate of 0.001, and report the accuracy and F1-score on the validation and test sets. Analyze any differences in performance, training stability, or computational efficiency between the manual nn.Linear approach and the optimized nn.RNN library call.

b. Standard RNNs often struggle to capture long-term dependencies due to vanishing gradients, which weaken the signal from early inputs as it propagates toward distant outputs. Since these models are practically limited in how much historical context they can retain, each review is truncated to a fixed length, where only the last max_len tokens are fed into the network. This max_len parameter is defined in the provided pre-processing template. You are encouraged to experiment with different values for max_len and observe how these variations impact the model's performance. (It is important to note that Truncated Backpropagation Through Time (TBPTT) is a technique for computational efficiency; it does not mitigate the underlying issue of information decay.)

4. Long-Short Term Memory (LSTM)



$$\boldsymbol{i}_t = \sigma\left(\boldsymbol{W}_i \begin{pmatrix} \boldsymbol{h}_{t-1} \\ \boldsymbol{x}_t \end{pmatrix} + \boldsymbol{b}_i\right)$$

$$\boldsymbol{f}_t = \sigma\left(\boldsymbol{W}_f \begin{pmatrix} \boldsymbol{h}_{t-1} \\ \boldsymbol{x}_t \end{pmatrix} + \boldsymbol{b}_f\right)$$

$$\boldsymbol{o}_t = \sigma\left(\boldsymbol{W}_o \begin{pmatrix} \boldsymbol{h}_{t-1} \\ x_t \end{pmatrix} + \boldsymbol{b}_o\right)$$

$$\boldsymbol{g}_t = tanh\left(\boldsymbol{W}_g \begin{pmatrix} \boldsymbol{h}_{t-1} \\ \boldsymbol{x}_t \end{pmatrix} + \boldsymbol{b}_g\right)$$

$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \boldsymbol{g}_t$$

$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh\left(\boldsymbol{c}_t\right)$$

Implement an LSTM architecture each featuring 128 hidden units followed by a 2-node fully connected output layer for binary sentiment classification. You are required to build the LSTM cell through both manual construction using nn.Linear layers and library implementation utilizing the nn.LSTM module as well. You may utilize the same learning rate and epoch count as the previous RNN models or perform hyperparameter tuning to optimize results. Furthermore, experiment with varying max_len values to evaluate the LSTM's effectiveness in capturing long-term dependencies and to observe how these changes impact the performance.

Resources:
You may find helpful tutorials on RNNs and LSTMs using Pytorch here:

1. RNN: https://www.geeksforgeeks.org/deep-learning/implementing-recurrent-neural-networks-in-pytorch/
2. LSTM: https://www.geeksforgeeks.org/deep-learning/long-short-term-memory-networks-using-pytorch/
3. https://colah.github.io/posts/2015-08-Understanding-LSTMs/
4. https://www.learnpytorch.io/00_pytorch_fundamentals/